模型架构升级：从 GNN 到 Transformer

为什么要替换 GNN?

原有架构使用图神经网络（GNN）编码已放置的粒子状态，存在几个明显问题：

1. 训练无法真正并行
GNN 需要邻接矩阵，而每个训练样本的粒子数不同，邻接矩阵大小也不同。训练时只能用 Python for 循环逐个计算图编码，256 个样本的 batch 实际上是串行的，GPU 利用率极低。

2. 邻接矩阵维护复杂
每放一个粒子就要增量更新邻接矩阵，代码复杂且容易出错。

3. 感受野受限
GNN 每层只能聚合直接相邻的粒子，需要多层才能感知全局结构。

完整架构图

```
                    环境状态输入
═══════════════════════════════════════════════════════════════

  已放置粒子 (N 个)                    候选放置点 (1000 个)

  ┌─────────────────┐                  ┌──────────────────────┐
  │  p₁: [x,y,z,r]  │                  │  c₁: [x,y,z,r,coord] │
  │  p₂: [x,y,z,r]  │                  │  c₂: [x,y,z,r,coord] │
  │      ...        │                  │        ...           │
  │  pₙ: [x,y,z,r]  │                  │c₁₀₀₀: [x,y,z,r,coord]│
  └─────────┬───────┘                  └──────────┬───────────┘
            │ 归一化: pos/L, r/r_max               │ shape: (1000, 5)
            │ shape: (N, 4)                        │
            ▼                                      ▼

  ┌─────────────────┐                  ┌──────────────────────┐
  │  Linear(4 → d)  │                  │  CandidateEncoder    │
  │  + ReLU         │                  │  MLP: 5→32→64→128    │
  └─────────┬───────┘                  └──────────┬───────────┘
            │ shape: (N, d)                        │ shape: (1000, 128)
            │                                      │
  ┌─────────▼───────┐                              │
  │  前置 [CLS]     │                              │
  │  token (1, d)   │                              │
  └─────────┬───────┘                              │
            │ shape: (N+1, d)                      │
            ▼                                      │
  ┌──────────────────────────────┐                │
  │      Transformer Encoder     │                │
  │  ┌────────────────────────┐  │                │
  │  │ Multi-Head Self-Attention │ │                │
  │  │ 每个 token 看到所有粒子  │  │                │
  │  └────────────┬───────────┘  │                │
  │               │              │                │
  │  ┌────────────▼───────────┐  │                │
  │  │   Feed-Forward Network │  │                │
  │  └────────────┬───────────┘  │                │
  │               │              │                │
  │        × L 层 (默认 3 层)    │                │
  └──────────────┬───────────────┘                │
                 │                                │
            取 CLS 位置输出                        │
```

```
          │ shape: (d,)                │
          ▼                            │
┌─────────────────┐                    │
│  Linear(d→128)  │                    │
└─────────────────┘                    │
          │                            │
          │ shape: (128,)              │
          │   "当前堆积需要什么？"        │
          │                            │
          │        CLS · cand_emb      │
          └──────────── . ─────────────┘
                        │
                (1000,) 原始分数
                        │
          ┌─────────────▼─────────────┐
          │  mask: 无效候选点 → -inf    │
          └───────────────────────────┘
                        │
          ┌─────────────▼─────────────┐
          │  softmax(scores / T)       │
          │  → 概率分布 (1000,)         │
          └───────────────────────────┘
                        │
                采样 / 取 argmax
                        │
                        ▼
                选中候选点 → 放置粒子
```

═══

训练时的 Batch 处理

```
Batch 内 B 个训练样本，粒子数各不相同：

  样本 1: N₁ = 12 个粒子    ▓▓▓
```

同步修复的问题

采集与训练的 temperature 不一致

采集时：

```
probs = softmax(scores / T)   # 用了 temperature
```

训练时原来：

```
log_probs = log_softmax(scores)   # T 硬编码为 1
```

修复后训练时统一除以 temperature，保证策略梯度计算的 log π(a|s) 与采集分布一致。

架构对比

	原 GNN 架构	新 Transformer 架构
粒子编码	GNN + 邻接矩阵	Self-Attention，无需邻接矩阵
训练并行	for 循环，串行	真正 batch，并行
候选点打分	MLP FusionDecoder	CLS 点积，直接高效
代码复杂度	高（邻接矩阵维护）	低
感受野	局部邻居	全局